Dylan Montgomery

Dr. En Cheng

Database Management 475/575

2 October 2026

Stage 3: Development Plan for Almanac

Team Lesotho is Dylan Montgomery (captain) and Andrew Styles. Our project, Almanac, is a farm record book where every planting, harvest, or sale is tied to a plant taxonomy and to Ohio produce auction prices from 2019 to 2026.

Database and Software Platform

We will use PostgreSQL 16 for the live site. It fully supports recursive queries and window functions, which our advanced functions depend on. We will keep using SQLite for local development, since the schema was first built there and only needs small syntax changes to move between the two.

The application is written in Python 3.12 with Flask and Jinja2 templates. We are writing the SQL by hand instead of using an ORM, and all user input is passed as query parameters to prevent SQL injection. Charts will use Chart.js. The data loaders are Python scripts using pdfplumber and BeautifulSoup. Code is shared on GitHub, and the site will be hosted on a Linux server with Gunicorn and Nginx.

Relational Schema

The database has twelve tables, fifteen foreign keys, and two views. The region and taxon tables point to themselves to form hierarchies. The first eight tables hold reference data we load ourselves, and the last four hold data that growers enter through the site.

```
CREATE TABLE observer (
    observer_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name         VARCHAR(120) NOT NULL UNIQUE,
    domain       VARCHAR(255) UNIQUE,
    granularity  VARCHAR(16)  NOT NULL
                 CHECK (granularity IN ('lot', 'product_day'))
);
```

```
CREATE TABLE region (
    region_id         INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    parent_region_id  INTEGER REFERENCES region (region_id),
    name              VARCHAR(120) NOT NULL,
    kind              VARCHAR(20)  NOT NULL
                      CHECK (kind IN ('country', 'state', 'county', 'other')),
    UNIQUE (parent_region_id, name),
    CHECK (parent_region_id <> region_id)
);
```

```
CREATE TABLE taxon (
    taxon_id         INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    parent_taxon_id  INTEGER REFERENCES taxon (taxon_id),
    path             VARCHAR(512) NOT NULL UNIQUE,
    depth            SMALLINT     NOT NULL CHECK (depth BETWEEN 0 AND 11),
    rank             VARCHAR(20)  NOT NULL,
    scientific_name  VARCHAR(200) NOT NULL,
    common_name      VARCHAR(200),
    CHECK (parent_taxon_id <> taxon_id)
);
```

```
CREATE TABLE product (
    product_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    slug        VARCHAR(80)  NOT NULL UNIQUE CHECK (slug = LOWER(slug)),
    title       VARCHAR(200) NOT NULL,
    taxon_id    INTEGER REFERENCES taxon (taxon_id) ON DELETE RESTRICT
);
```

```
CREATE TABLE product_alias (
    label_norm    VARCHAR(255) PRIMARY KEY,
    product_id    INTEGER   NOT NULL
                  REFERENCES product (product_id) ON DELETE CASCADE,
    match_rung    SMALLINT  NOT NULL CHECK (match_rung BETWEEN 0 AND 4),
    confirmed_at  TIMESTAMP NOT NULL
);
```

```
CREATE TABLE market_lot (
    lot_id       INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    observer_id  INTEGER NOT NULL
                 REFERENCES observer (observer_id) ON DELETE RESTRICT,
    sale_date    DATE    NOT NULL CHECK (sale_date >= DATE '2019-01-01'),
    raw_label    VARCHAR(255) NOT NULL,
    label_norm   VARCHAR(255) NOT NULL,
    unit         VARCHAR(40),
    quantity     NUMERIC(10,2) CHECK (quantity > 0),
    low_cents    INTEGER CHECK (low_cents  >= 0),
    high_cents   INTEGER CHECK (high_cents >= 0),
    avg_cents    INTEGER CHECK (avg_cents  >= 0),
    CHECK (low_cents <= high_cents),
    CHECK (avg_cents BETWEEN low_cents AND high_cents)
);
CREATE INDEX ix_lot_label    ON market_lot (label_norm);
CREATE INDEX ix_lot_date     ON market_lot (sale_date);
CREATE INDEX ix_lot_obs_date ON market_lot (observer_id, sale_date);
```

```
CREATE TABLE market_lot_quarantine (
    quarantine_id   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    observer_id     INTEGER NOT NULL
                    REFERENCES observer (observer_id) ON DELETE RESTRICT,
    sale_date       DATE    NOT NULL,
    raw_label       VARCHAR(255) NOT NULL,
    unit            VARCHAR(40),
    quantity        NUMERIC(10,2),
    low_cents       INTEGER,
    high_cents      INTEGER,
    avg_cents       INTEGER,
    reason          VARCHAR(255) NOT NULL,
    quarantined_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE event_kind (
    event_kind_code  VARCHAR(20) PRIMARY KEY,
    label            VARCHAR(60) NOT NULL UNIQUE,
    is_yield         BOOLEAN     NOT NULL DEFAULT FALSE
);
```

```
CREATE TABLE farm (
    farm_id      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name         VARCHAR(200) NOT NULL,
    domain       VARCHAR(255) UNIQUE,
    region_id    INTEGER NOT NULL
                 REFERENCES region (region_id) ON DELETE RESTRICT,
    owner_email  VARCHAR(255) UNIQUE,
    created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE plot (
    plot_id    INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    farm_id    INTEGER NOT NULL
               REFERENCES farm (farm_id) ON DELETE CASCADE,
    name       VARCHAR(100) NOT NULL,
    area_sq_m  NUMERIC(10,2) CHECK (area_sq_m > 0),
    UNIQUE (farm_id, name)
);
```

```
CREATE TABLE crop_catalog (
    farm_id   INTEGER NOT NULL
              REFERENCES farm (farm_id) ON DELETE CASCADE,
    taxon_id  INTEGER NOT NULL
              REFERENCES taxon (taxon_id) ON DELETE RESTRICT,
    added_on  DATE NOT NULL DEFAULT CURRENT_DATE,
    notes     VARCHAR(500),
    PRIMARY KEY (farm_id, taxon_id)
);
```

```
CREATE TABLE farm_event (
    event_id         INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    farm_id          INTEGER NOT NULL
                     REFERENCES farm (farm_id) ON DELETE CASCADE,
    plot_id          INTEGER
                     REFERENCES plot (plot_id) ON DELETE SET NULL,
    taxon_id         INTEGER NOT NULL
                     REFERENCES taxon (taxon_id) ON DELETE RESTRICT,
    event_kind_code  VARCHAR(20) NOT NULL
                     REFERENCES event_kind (event_kind_code) ON DELETE RESTRICT,
    product_id       INTEGER
                     REFERENCES product (product_id) ON DELETE SET NULL,
    event_date       DATE NOT NULL,
    quantity         NUMERIC(10,2) CHECK (quantity >= 0),
    unit             VARCHAR(40),
    notes            VARCHAR(1000),
    CHECK (quantity IS NULL OR unit IS NOT NULL)
);
CREATE INDEX ix_event_farm_date ON farm_event (farm_id, event_date);
CREATE INDEX ix_event_taxon     ON farm_event (taxon_id);
```

Deleting a plot keeps its events and just clears the plot_id, while a taxon that has events recorded against it cannot be deleted. The two views below are used by most of our price queries and to track how many auction lots have been matched to a product.

```
CREATE VIEW v_priced_lot AS
SELECT l.lot_id, l.sale_date, o.name AS source,
       p.product_id, p.title, p.taxon_id,
       l.unit, l.quantity,
       l.low_cents / 100.0 AS low_price,
       l.high_cents / 100.0 AS high_price,
       l.avg_cents / 100.0 AS avg_price
FROM market_lot l
JOIN observer o       ON o.observer_id = l.observer_id
JOIN product_alias a  ON a.label_norm = l.label_norm
JOIN product p        ON p.product_id = a.product_id;

CREATE VIEW v_resolution_coverage AS
SELECT o.name AS source,
       COUNT(*) AS total_lots,
       COUNT(a.label_norm) AS resolved_lots,
       ROUND(100.0 * COUNT(a.label_norm) / COUNT(*), 1) AS pct_resolved
FROM market_lot l
JOIN observer o ON o.observer_id = l.observer_id
LEFT JOIN product_alias a ON a.label_norm = l.label_norm
GROUP BY o.name;
```

Sources of Real Data

The price data is already collected and loaded. It comes from the published sale reports of County Line Produce Auction, Mt. Hope Auction, the USDA AMS Market News report for Mount Hope, and Geauga Growers Produce Auction, covering 560 trading days and 47,516 lots. Our loader reads each report, pulls out the date, item, unit, quantity, and prices, and inserts the rows. Rows that fail a price check go into the quarantine table instead.

The plant taxonomy (4,118 entries) was built from USDA PLANTS and GBIF. We made the product list (217 entries) ourselves by grouping the different ways the auctions describe the same item. The farm list (238 farms) comes from public listings of Ohio farms.

Plots, crop catalogs, and events will come from real users. We will have the site live at least a week before the final presentation, let Dr. Cheng know, and ask classmates and local growers to use it.

Division of Tasks

Dylan is handling the stage reports, the database and data loading, the first two advanced functions (the taxonomy price roll-up and the crop decision engine), and putting the site online.

Andrew is handling the Flask pages, the insert, update, and delete forms, the search pages, and the third advanced function (matching auction labels to products). We will both test, review each other's code, and work on the presentation and final report.

Timeline and Milestones

Week of October 5: Submit Stage 3. Set up PostgreSQL and load the data. Start the Flask app.

Week of October 12: Insert, update, and delete pages. Search pages.

Week of October 19: Price chart and the multi-table join page. Stage 4 demo.

Week of October 26: Stage 4 report. Taxonomy price roll-up.

Week of November 2: Crop decision engine and label matching.

Week of November 9: Stage 5 report. Put the site online and invite users.

Week of November 16: Collect user data and fix bugs.

Week of November 23: Final testing and presentation practice.

Week of November 30: Stage 6 final demo.

Week of December 7: Stage 7 final submission.